EMBEDDED CYBERSECURITY, IOT WIRELESS SYSTEMS & USB HID RESEARCH REPORT

ESP32-S3 Phone-Controlled USB HID Keyboard and Mouse with BadUSB Simulation and Defensive Keystroke Firewall

Native Composite TinyUSB Emulation, Dual-Mode Wi-Fi Mobile Web Controller, Deterministic Scenario State Machine & Win32 Hook Defense

Project Identity & Academic Attribution

Authors:	**Ram Karthik G & Harini V**	**Roll Number:**	CH.SC.U4CYS23037 & CH.SC.U4CYS23010
Department:	Cybersecurity	**Institution:**	Amrita Vishwa Vidyapeetham, Chennai

Executive Summary & Architectural Overview

Modern operating systems implicitly trust USB Human Interface Devices (HID) the moment they enumerate on the host bus. While this zero-configuration trust model enables universal plug-and-play usability, it exposes a severe physical-layer attack vector known as BadUSB, in which a programmable microcontroller impersonates a composite keyboard and mouse to inject rapid OS commands. Conversely, legitimate wireless remote input solutions typically require installing proprietary receiver software on the host PC, limiting deployment on locked-down laboratory or presentation workstations.

This project engineers and validates a unified offensive-and-defensive embedded cybersecurity platform built on the Espressif ESP32-S3 SoC and a companion Windows Host Keystroke Firewall. Using the native USB OTG peripheral (GPIO 19 D- / GPIO 20 D+) and the TinyUSB stack, the ESP32-S3 enumerates simultaneously as a composite Boot Keyboard and 3-button Scroll Mouse while hosting a responsive dark-mode Single-Page Application (SPA) from flash memory (PROGMEM) over dual-mode Wi-Fi (WIFI_AP_STA). An operator using an Android smartphone can launch or abort four non-destructive security awareness scenarios, type on a full 6-row virtual keyboard with hardware modifier latching, control the host cursor via a 60 Hz multi-touch trackpad, and demonstrate real-time host mitigation using a Python WH_KEYBOARD_LL low-level input suppression firewall.

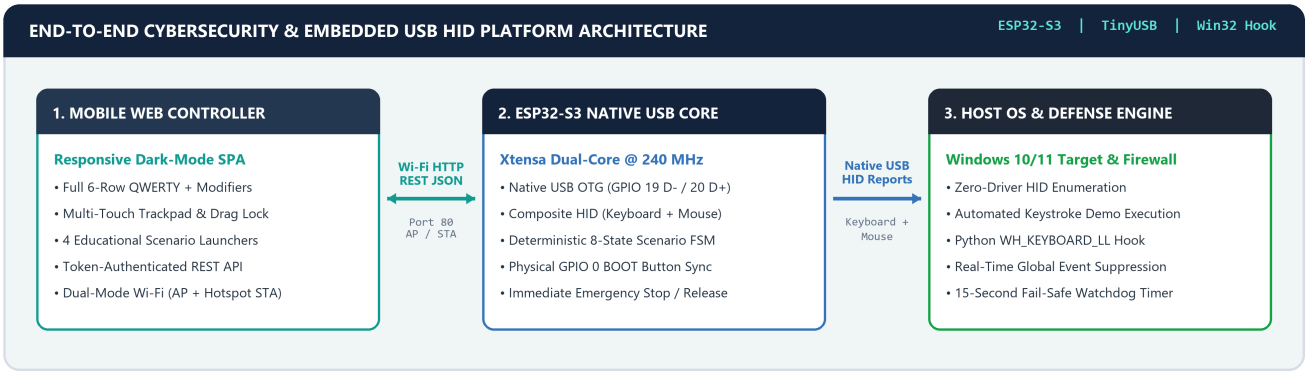


Figure 1.1: End-to-End Platform Architecture bridging the Mobile Web SPA, ESP32-S3 Native Composite TinyUSB Core, and Windows Host Firewall.

Table 1.1: Summary of Core Platform Subsystems, Implementation Mechanisms, and Verified Metrics

Subsystem Domain	Implemented Engineering Mechanism	Verified Hardware / QA Metric
Composite USB HID	TinyUSB USBHIDKeyboard + USBHIDMouse on GPIO 19/20	Zero-driver enumeration; 3.5 s boot guard; [-127, 127] clamping
Dual-Mode Wireless	WIFI_AP_STA (192.168.4.1 SoftAP + 2.4 GHz Hotspot STA) + mDNS	Live phone access via 10.85.248.29 & http://badusb.local
Scenario Engine FSM	8-state non-blocking FSM + portMUX_TYPE spinlock mailbox	Auto-runs Scenario 1 on boot; GPIO 0 cycles 1->2->3->4->1
Remote Keyboard & Mouse	4 keyboard sub-views, latched modifiers, 60 Hz multi-touch pad	1-finger move/tap, 2-finger scroll/R-click, drag lock, 256B guard
Emergency Abort Safety	Cooperative 10 ms smartDelay polling + dual HID releaseAll()	<= 10 ms abort latency; zero stuck modifiers or mouse buttons
Host Keystroke Defense	Python WH_KEYBOARD_LL hook (suppress=True) + 15 s watchdog	Blocks injected keys globally; auto-disarms after 15.0 seconds
Firmware & QA Validation	Compiled for esp32:esp32:esp32s3 (hwcdc); 3 automated test suites	851,520 B Flash (64.9%), 59,496 B RAM (18.1%); 22/22 tests pass

SECTION 01 | ABSTRACT, PROBLEM STATEMENT & OBJECTIVES

1.1 Background and the USB HID Trust Model

Universal Serial Bus (USB) Human Interface Devices represent the primary physical trust boundary between human operators and modern computing platforms. When a peripheral is connected to a USB host port, the operating system reads its USB Device, Configuration, Interface, and HID Report Descriptors. Whenever an interface declares USB Class 0x03 (HID), Subclass 0x01 (Boot Interface), and Protocol 0x01 (Keyboard) or 0x02 (Mouse), the host OS kernel automatically binds built-in class drivers (kbdclass.sys and mouclass.sys on Windows) without requiring driver signatures, user confirmation dialogs, or cryptographic device authentication.

As first formalized by Nohl and Lell in their seminal BadUSB research, the operating system has no native architectural mechanism at the USB HID layer to distinguish electrical scancodes generated by a physical mechanical switch from synthetic HID reports generated by an embedded microcontroller timer. A microcontroller capable of native USB device emulation can therefore open command shells (`Win + R`), execute administrative utilities, or manipulate cursor coordinates at hundreds of characters per second.

1.2 Problem Statement & Academic Motivation

Despite the pedagogical importance of demonstrating physical USB security risks in university cybersecurity curricula, existing hardware tools exhibit three critical engineering shortcomings that hinder safe classroom instruction and practical multi-role use:

- **Blind, Non-Interruptible Static Execution:** Conventional BadUSB demonstration boards execute hardcoded keystroke arrays immediately upon USB insertion. They provide no wireless status feedback, no runtime scenario switching, and no emergency abort mechanism if the target window fails to open in time.
- **Host Agent Dependency in Wireless Input Tools:** Standard smartphone-to-PC remote keyboard and mouse applications require installing a background TCP/UDP server daemon on the target computer. In restricted academic laboratories, smart classrooms, or hardware debugging benches, installing third-party host agents is disallowed.
- **Lack of Integrated Defensive Countermeasures:** Demonstrating offensive keystroke injection without a companion host-side mitigation leaves students without actionable defensive engineering patterns, while naive keyboard hooks risk locking out the instructor's physical keyboard if not paired with an automated fail-safe timer.

1.3 Core Research and Engineering Objectives

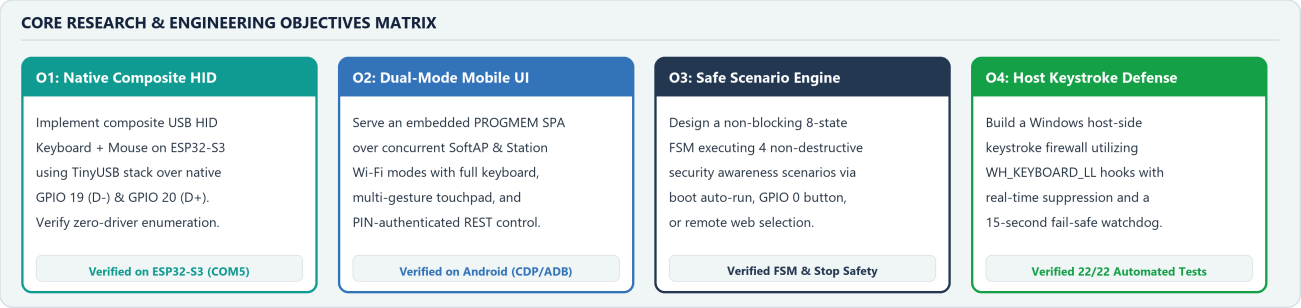


Figure 1.2: Four-Pillar Research and Engineering Objectives Matrix with Verified Hardware & Test Milestones.

1.4 Scope, Safety Boundaries, and Ethical Design

This system is strictly scoped as an authorized educational demonstration platform, wireless driverless HMI peripheral, and defensive endpoint security testbed. Rigorous engineering safeguards are enforced across both firmware and host software:

- **Non-Destructive Educational Scenarios Only:** None of the four embedded scenarios delete files, modify system registries, exfiltrate credentials, or establish persistence. Scenario 3 displays an educational awareness notice inside Notepad, and Scenario 4 lists saved Wi-Fi SSID names (`netsh wlan show profiles`) without reading key material.
- **Credential Redaction & Input Bounding:** All Wi-Fi station passphrases, SoftAP passwords, authentication PINs, and device serial numbers are redacted as `[REDACTED]` in documentation and masked in the mobile UI (••••••••). Bulk text transmission is bounded to `MAX_TEXT_LENGTH = 256` characters per request.

ARCHITECTURAL CONVERGENCE: By combining a composite TinyUSB HID engine, an embedded PROGMEM web application, a deterministic 8-state scenario FSM, and a Win32 WH_KEYBOARD_LL firewall within a single repository, the platform serves simultaneously as a daily-use wireless mobile keyboard/trackpad and a controlled cybersecurity lab.

SECTION 02 | SYSTEM ARCHITECTURE & TECHNOLOGY STACK

2.1 End-to-End Three-Tier System Architecture

The platform architecture is partitioned into three decoupled tiers that communicate over standard protocols: (1) the Android Phone Controller Tier (rendering the Single-Page Application over HTTP REST JSON), (2) the ESP32-S3 Embedded Firmware Tier (managing dual-mode Wi-Fi, token validation, the thread-safe Scenario FSM, and composite TinyUSB HID report generation), and (3) the Windows Host & Firewall Tier (enumerating the composite USB device and filtering input events via the Win32 low-level keyboard hook chain).

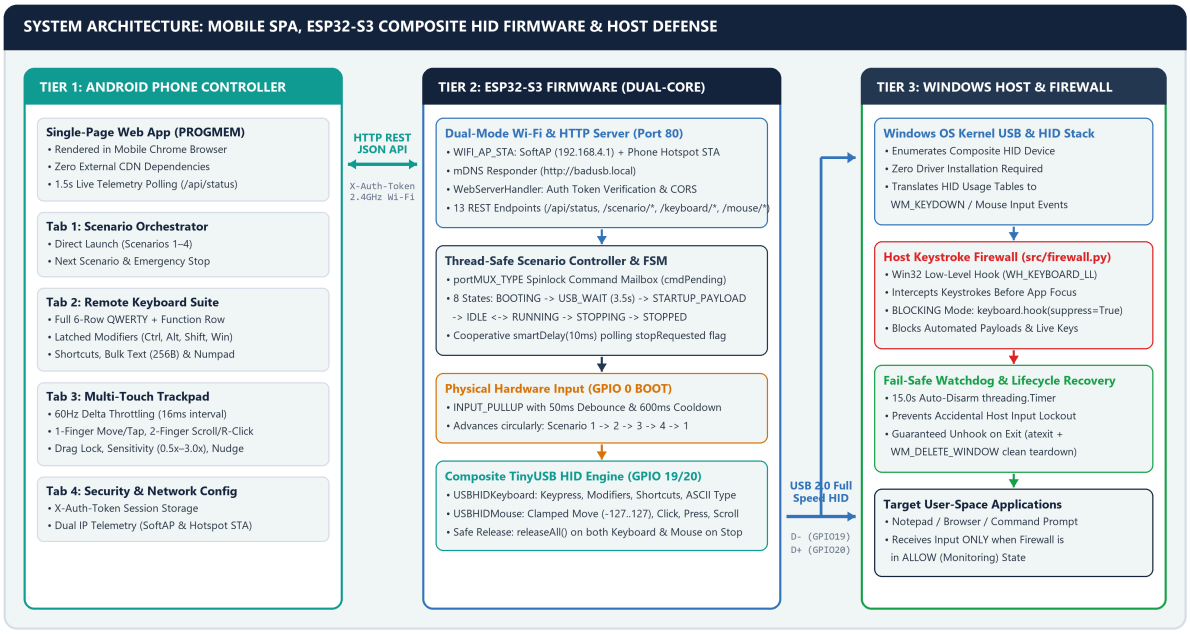


Figure 2.1: Complete Three-Tier System Architecture illustrating Mobile SPA Tabs, ESP32-S3 Firmware Modules, and Windows Host Firewall Pipeline.

2.2 Dual-Mode Wi-Fi Topology and Native USB OTG Interfacing

A foundational hardware distinction of the Espressif ESP32-S3 over legacy ESP32-WROOM modules is its integrated full-speed (12 Mbps) USB 2.0 On-The-Go (OTG) peripheral controller routed directly to GPIO 19 (**`USB D-`**) and GPIO 20 (**`USB D+`**). When compiled with Arduino-ESP32 TinyUSB configuration flags (**`USBMode=hwcdc,CDCOnBoot=cdc`**), the ESP32-S3 presents a composite USB descriptor over a single USB data cable that simultaneously binds three host interfaces: USB CDC Serial (**`COM5`** for telemetry and flashing), a standard 104-key USB HID Boot Keyboard, and a 3-button USB HID Mouse with vertical scroll.

On the wireless side, **`WebServerHandler::begin()`** configures the IEEE 802.11 b/g/n radio in concurrent **`WIFI_AP_STA`** mode. The SoftAP interface immediately establishes an isolated control network (**`SSID: BadUSB-Controller`**, Passphrase: **`[REDACTED]`**, Static IP: **`192.168.4.1`**), while the Station interface connects automatically to the user's 2.4 GHz mobile hotspot (**`SSID: [REDACTED]`**, Passphrase: **`[REDACTED]`**, assigned DHCP IP **`10.85.248.29`**). Because the smartphone hosts the 2.4 GHz hotspot, the phone browser can communicate directly with **`http://10.85.248.29`** (or **`http://badusb.local`** via mDNS) without disconnecting from mobile cellular data.

Table 2.1: Repository Module Breakdown and Architectural Responsibilities Verified Against Source Code

Architectural Layer	File / Module	Key Classes, Libraries & Technical Responsibilities
Main Firmware Entry	badusb_firmware.ino	Initializes Serial (115200), ScenarioController, WebServerHandler, and cooperative loop()
System Configuration	config.h	Defines WIFI_AP/STA credentials [REDACTED], AUTH_PIN [REDACTED], GPIO 0 pin, and timing guards
HID Keyboard Engine	keyboard_controller.h/.cpp	Wraps USBHIDKeyboard; ASCII typing, named keys, modifier bitmasks, shortcuts, releaseAll()
HID Mouse Engine	mouse_controller.h/.cpp	Wraps USBHIDMouse; clamped [-127, 127] relative move, L/R/M click, drag press/release, scroll
Scenario State Machine	scenario_controller.h/.cpp	8-state FSM, portMUX_TYPE spinlock command mailbox, GPIO 0 debounce, Scenarios 1–4
HTTP REST Server	web_server_handler.h/.cpp	Port 80 WebServer, WIFI_AP_STA setup, X-Auth-Token checks, CORS preflight, 13 API routes
Embedded Mobile SPA	web_assets.h (PROGMEM)	Responsive dark-mode HTML5/CSS3/JS interface: Scenarios, 4-View Keyboard, Touchpad, Settings
Host Keystroke Firewall	src/firewall.py & firewall.py	Python 3.12 Tkinter GUI, keyboard.hook(suppress=True) WH_KEYBOARD_LL filter, 15s Timer

SECTION 03 | EMBEDDED FIRMWARE & SCENARIO EXECUTION ENGINE

3.1 Composite USB Initialization Ordering & Enumeration Guard

TinyUSB requires all HID report descriptors to be registered before the physical USB bus attaches to the host. Inside **`ScenarioController::begin()`**, the firmware calls **`keyboard.begin()`** (**`USBHIDKeyboard::begin()`**) and **`mouse.begin()`** (**`USBHIDMouse::begin()`**) prior to invoking **`USB.begin()`**. Once **`USB.begin()`** asserts the D+ pull-up resistor on GPIO 20, the host OS initiates USB enumeration. Because Windows takes 1.5 to 3.0 seconds to complete descriptor negotiation and ready the HID input queues, the FSM enters **`STATE_USB_WAIT`** and enforces a non-blocking 3,500 ms stabilization window (**`USB_READY_WAIT_MS = 3500`**) before permitting any automated or remote HID transmission.

3.2 Deterministic 8-State Finite State Machine & Spinlock Synchronization

To guarantee predictable behavior across boot-time execution, physical button presses, and asynchronous web requests, **`ScenarioController`** implements an 8-state FSM (**`STATE_BOOTING`**, **`STATE_USB_WAIT`**, **`STATE_STARTUP_PAYLOAD`**, **`STATE_IDLE`**, **`STATE_RUNNING`**, **`STATE_STOPPING`**, **`STATE_STOPPED`**, and **`STATE_ERROR`**). External triggers never mutate state variables directly; instead, they queue a command (**`CMD_SELECT_SCENARIO`**, **`CMD_NEXT_SCENARIO`**, or **`CMD_STOP`**) inside an ESP-IDF FreeRTOS critical section (**`portENTER_CRITICAL(&mux)`** / **`portEXIT_CRITICAL(&mux)`**). When the 3,500 ms boot wait completes, **`STATE_STARTUP_PAYLOAD`** automatically runs Scenario 1 once (**`startupPending = false; startupDone = true`**) and transitions to **`STATE_IDLE`**.

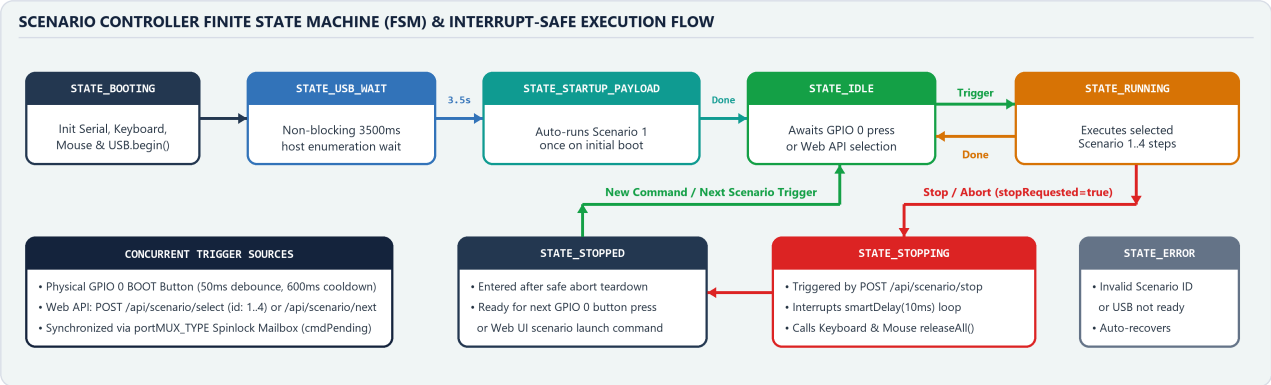


Figure 3.1: ScenarioController 8-State Finite State Machine (FSM) and Interrupt-Safe Execution Flow.

3.3 Physical GPIO 0 BOOT Button Advancement & Cooperative Abort Safety

The ESP32-S3 onboard **`BOOT`** button (**`GPIO 0`**, active-low with internal pull-up) is polled on every **`ScenarioController::update()`** cycle. Raw readings pass through a 50 ms debounce filter (**`BUTTON_DEBOUNCE_MS = 50`**) and a 600 ms post-trigger cooldown (**`BUTTON_COOLDOWN_MS = 600`**). Pressing the button invokes **`requestNextScenario()`**, which computes **`(currentScenario % 4) + 1`**, advancing circularly through Scenarios **`1 -> 2 -> 3 -> 4 -> 1`**.

During scenario execution, all inter-step pauses use **`ScenarioController::smartDelay(unsigned long ms)`**. Rather than blocking the CPU, **`smartDelay`** loops in 10 ms increments (**`STEP_POLL_INTERVAL_MS = 10`**), invoking the registered client callback (**`webHandler.handleClient()`**) and checking **`stopRequested`** on every iteration. During live hardware QA, a critical deadlock defect was discovered and resolved in **`requestStop()`**: calling **`keyboard.releaseAll()`** inside **`portENTER_CRITICAL(&mux)`** previously stalled the TinyUSB task while interrupts were disabled. Moving **`keyboard.releaseAll()`** and **`mouse.releaseAll()`** outside the spinlock critical section achieved instantaneous, crash-free aborts.

Table 3.1: Verified Catalog of Non-Destructive Educational Scenarios Implemented in scenario_controller.cpp

ID	Scenario Title	Exact HID Sequence Executed on Host	Security Awareness Objective
1	Notepad Awareness (Boot Auto-Run)	GUI+r (Win+R) -> wait 750ms -> type 'notepad' + Enter -> wait 1200ms -> type 3-line security warning message	Demonstrates how unattended USB ports allow immediate process creation and text injection.
2	Portal Redirection	GUI+r (Win+R) -> wait 750ms -> type 'cmd' + Enter -> wait 1000ms -> type 'start https://intranet.cb.amrita.edu/' + Enter	Demonstrates forced browser navigation and potential credential phishing via command shell.
3	Ransomware Awareness	GUI+r (Win+R) -> wait 750ms -> type 'notepad' + Enter -> wait 1200ms -> type '[SECURITY EDUCATION NOTICE]' banner	Safely simulates rapid payload staging visibility in a classroom without modifying user files.
4	Wi-Fi Reconnaissance	GUI+r (Win+R) -> wait 750ms -> type 'cmd' + Enter -> wait 1000ms -> execute 'netsh wlan show profiles' in console	Shows how local system enumeration commands can list saved wireless network profile names.

SECTION 04 | WI-FI PHONE CONTROLLER & REMOTE KEYBOARD

4.1 Embedded PROGMEM Single-Page Application & Live Telemetry

The mobile web interface is compiled directly into the firmware binary inside **`badusb_firmware/web_assets.h`** as a **`PROGMEM`** string literal (**`INDEX_HTML`**). By avoiding external CSS frameworks or CDN scripts, the interface loads instantaneously over both the standalone SoftAP (**`192.168.4.1`**) and the phone's 2.4 GHz hotspot (**`10.85.248.29`**). During live Android QA on the Vivo V50e (**`chrome://inspect`** over ADB), an unclosed **`<div id="tabKeyboard">`** container bug that previously hid the Mouse and Settings tabs was identified and fixed, along with adding explicit CORS **`HTTP_OPTIONS`** preflight support (**`204 No Content`**).

4.2 Token-Based Authentication and REST API Command Flow

To prevent unauthorized users on the same Wi-Fi subnet from injecting keystrokes, all state-modifying **`POST`** routes require authentication. In the Settings tab, the operator inputs the configured security PIN (**`[REDACTED]`**), which is verified via **`POST /api/auth/login`** against **`AUTH_PIN`** in **`config.h`**. Once authenticated, the client attaches **`X-Auth-Token: [REDACTED]`** to all **`fetch()`** requests. **`WebServerHandler::isAuthorized()`** validates this header before executing any scenario, keyboard, or mouse action, returning HTTP **`401 Unauthorized`** if missing or invalid.

4.3 Four-Sub-View Remote Keyboard, Modifiers & Shortcuts

The Keyboard tab (**`#tabKeyboard`**) replicates a complete desktop keyboard on the smartphone touchscreen through a persistent Hardware Modifier bar (**`Ctrl`**, **`Alt`**, **`Shift`**, **`Win`** with **`Auto-release modifier`** and **`Release All`** safety controls) and four specialized sub-views: (1) **`Full Layout`** (6-row desktop matrix including **`Esc`**, **`F1`**–**`F12`**, **`Del`**, symbol/number row, **`Tab`**, **`Caps Lock`**, **`Shift`**, **`Win`**, **`Alt`**, **`Menu`**, and **`Spacebar`**), (2) **`Shortcuts`** (one-tap macros such as **`Copy`**, **`Paste`**, **`Task Manager`**, **`Win+R`**, **`Win+D`**, **`Win+L`**, **`Alt+F4`**, and an inverted-T arrow/page navigation cluster), (3) **`Text & Live`** (256-character bulk string transmitter and real-time per-keystroke live input box), and (4) **`Numpad`** (17-key numeric keypad).

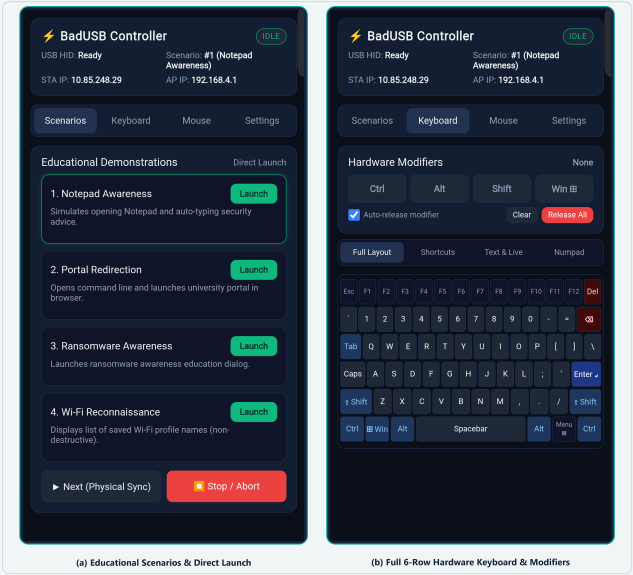


Figure 4.1: Verified Live Android Phone Screenshots — (a) Educational Scenarios & Direct Launch Tab; (b) Full 6-Row Hardware Keyboard & Modifier Bar.

Table 4.1: Core Status, Authentication, Scenario, and Keyboard REST API Endpoints (web_server_handler.cpp)

HTTP Method & Route	Auth	Request Payload Parameters	Firmware Handler & HID Action
GET /api/status	Public	None	Returns JSON: usbReady, state, currentScenario, staIp, apIp, authRequired
POST /api/auth/login	Public	pin / token ([REDACTED])	Verifies credential against AUTH_PIN; returns 200 OK or 401 Unauthorized
POST /api/scenario/select	Token	id (integer 1..4)	Queues CMD_SELECT_SCENARIO in spinlock mailbox to launch Scenario 1–4
POST /api/scenario/next	Token	None	Queues CMD_NEXT_SCENARIO; advances circularly 1 -> 2 -> 3 -> 4 -> 1
POST /api/scenario/stop	Token	None	Sets stopRequested=true, aborts smartDelay(), calls Keyboard/Mouse releaseAll()
POST /api/keyboard/key	Token	key (e.g. ENTER, F5, a)	Maps key name to USB HID usage code via KeyboardController::sendKeyByName()
POST /api/keyboard/shortcut	Token	mod (CTRL+SHIFT), key (ESC)	Presses modifier bitmask + target key for 25 ms, then releases cleanly
POST /api/keyboard/text	Token	text (<=256B), enter (0/1)	Streams printable ASCII characters via Keyboard.print() + optional Enter

SECTION 05 | REMOTE MOUSE CONTROL & HOST-SIDE DEFENSE

5.1 Multi-Touch Relative Trackpad, Gestures & Precision Nudge

The Mouse tab (`#tabMouse`) implements a low-latency wireless touchpad backed by `MouseController` (`mouse_controller.h/.cpp`). Touch coordinates on `#trackpadArea` are sampled via `touchstart`, `touchmove`, and `touchend` listeners throttled to 16 ms intervals (~60 Hz). Single-finger movement calculates relative (dx, dy) offsets scaled by the Cursor Sensitivity slider (`0.5x–3.0x`, default `1.8x`) and clamped to signed 8-bit HID integers (`[-127, 127]`) inside `MouseController::clampDelta()`. A single-finger tap (<10 px movement within 220 ms) triggers a Left Click, while a two-finger tap (<260 ms) triggers a Right Click and a two-finger vertical drag emulates the scroll wheel (`POST /api/mouse/scroll`). Dedicated buttons provide Left, Right, Double, and Middle Click, alongside a stateful `Drag Lock` toggle (`POST /api/mouse/press` and `/release`) and a Precision Nudge pad (`5px`, `20px`, `60px` step selector).

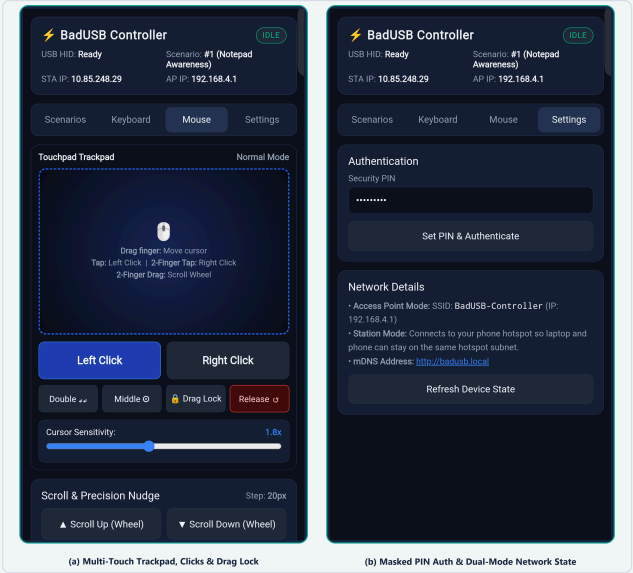


Figure 5.1: Verified Live Android Phone Screenshots — (a) Multi-Touch Trackpad, Click Bar & Drag Lock; (b) Masked PIN Auth & Dual-Mode Network Telemetry.

5.2 Windows Host-Side Keystroke Firewall Architecture

To demonstrate practical host-side defense against unauthorized HID keystroke injection, the platform includes a Python/Tkinter desktop security utility (`src/firewall.py`). When toggled from `ALLOW (Monitoring)` to `BLOCKING (Armed)` mode, `KeystrokeFirewall.enable_blocking()` installs a Win32 low-level keyboard hook (`WH_KEYBOARD_LL`) with `keyboard.hook(self._on_key_event, suppress=True)`. Because `WH_KEYBOARD_LL` executes in the global OS input chain prior to foreground window routing, returning with `suppress=True` drops all incoming keystrokes from both automated BadUSB scenarios and remote typing. To guarantee that the operator is never permanently locked out of their PC, arming the hook starts a 15.0-second watchdog `threading.Timer(15.0, self._auto_disarm)` and registers `atexit` / `WM_DELETE_WINDOW` cleanup handlers that invoke `keyboard.unhook_all()`.

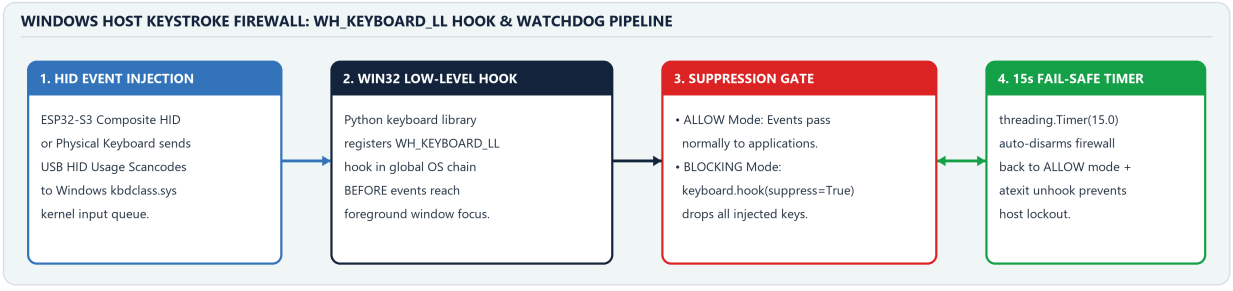


Figure 5.2: Windows Host Keystroke Firewall Interception Pipeline (WH_KEYBOARD_LL Hook, Suppression Gate, and 15s Fail-Safe Watchdog).

Table 5.1: Architectural Tradeoffs Between User-Space Global Hook Suppression and Kernel USB Device Filtering

Comparison Criterion	Implemented Host Firewall (WH_KEYBOARD_LL)	Hardware / Kernel USB Device Filtering
Operating System Layer	User-Space Win32 Hook Chain (SetWindowsHookEx)	Kernel-Mode USB Bus / Filter Driver (WDF / udev)
Suppression Granularity	Global: Suppresses all keyboard scancodes system-wide	Per-Device: Filters by USB Vendor/Product ID or port
Resilience to VID/PID Spoofing	High: Blocks keystrokes even if BadUSB spoofs Dell/Logitech IDs	Low: Bypassed when attacker clones an authorized VID/PID
Fail-Safe Lockout Protection	Built-in 15.0s auto-disarm watchdog + clean exit unhook	Requires safe-mode boot if primary keyboard is blacklisted

SECTION 06 | TESTING, RESULTS & TECHNICAL VALIDATION

6.1 Multi-Tier Verification Methodology (Simulation, Unit & Live Hardware)

A rigorous three-stage verification methodology was executed to validate every firmware module, mobile UI view, REST endpoint, and host firewall behavior. First, behavioral simulation and unit test suites (**`tests/test_controller_simulation.py`** and **`tests/test_firewall.py`**) verified the 8-state FSM, circular scenario progression, token authentication, mouse coordinate clamping, and **`WH_KEYBOARD_LL`** suppression/watchdog timers. Second, the firmware was compiled via **`arduino-cli`** (**`esp32:esp32:esp32s3:USBMode=hwcdc,CDCOnBoot=cdc`**) and flashed over **`COM5`** (**`851,520 bytes`**). Third, an Android phone (**`Vivo V50e`**, Android 16, serial **`[REDACTED]`**) was connected over ADB and inspected via Chrome DevTools Protocol (**`tests/test_live_phone_api.py`**), executing 10 live end-to-end API and DOM verification tests directly from the mobile browser against the physical ESP32-S3 at **`http://10.85.248.29/`**.

Table 6.1: Complete Automated & Live Hardware Verification Matrix (22 / 22 Tests Passed)

Test Suite	Automated Test Case Identifier	Verified Engineering Requirement & Behavior	Status
Controller Sim	`test_boot_runs_scenario_1_automatically`	3.5s USB enumeration guard + Scenario 1 auto-execution on boot	**PASS**
Controller Sim	`test_button_cycles_1_to_4_and_wraps`	GPIO 0 button advances circularly 1 -> 2 -> 3 -> 4 -> 1	**PASS**
Controller Sim	`test_direct_scenario_selection_and_validation`	Direct launch of Scenarios 1..4; HTTP 400 on invalid ID 99	**PASS**
Controller Sim	`test_stop_interrupts_and_releases_keys`	Emergency stop aborts running scenario & calls releaseAll()	**PASS**
Controller Sim	`test_authentication_protection`	Rejects unauthenticated POST (401); accepts valid X-Auth-Token	**PASS**
Controller Sim	`test_keyboard_text_and_special_keys`	ASCII typing, 256-char limit enforcement, and named special keys	**PASS**
Controller Sim	`test_shortcuts_and_modifier_release`	Composite CTRL+c shortcut press/release and modifier clear	**PASS**
Controller Sim	`test_mouse_move_click_drag_and_scroll`	Delta clamping [-127, 127], L/R/M click, drag lock, scroll wheel	**PASS**
Host Firewall	`test_initial_state_is_allow`	Firewall initializes in ALLOW state with no active keyboard hook	**PASS**
Host Firewall	`test_enable_blocking_installs_suppressive_hook`	Arms WH_KEYBOARD_LL hook with suppress=True + 15s watchdog	**PASS**
Host Firewall	`test_disable_blocking_unhooks_cleanly`	Disarms hook cleanly and cancels active threading.Timer	**PASS**
Host Firewall	`test_auto_disarm_safety_timeout`	Watchdog callback automatically restores ALLOW mode after 15.0s	**PASS**
Live Phone API	`test_01_ui_tabs_and_containers_present`	Phone Chrome DOM renders all 4 main tabs & 4 keyboard sub-tabs	**PASS**
Live Phone API	`test_02_status_endpoint_live`	Live GET /api/status returns usbReady:true, staIp:10.85.248.29	**PASS**
Live Phone API	`test_03_auth_enforcement`	Live unauthenticated POST from phone receives HTTP 401	**PASS**
Live Phone API	`test_04_auth_login_valid`	Live POST /api/auth/login with valid PIN returns 200 OK	**PASS**
Live Phone API	`test_05_keyboard_special_keys_and_modifiers`	Live phone dispatch of SPACE, BACKSPACE, CTRL+c, and release	**PASS**
Live Phone API	`test_06_keyboard_bulk_text_validation`	Live bulk text injection (200) and >256 char rejection (400)	**PASS**
Live Phone API	`test_07_mouse_move_click_drag_scroll_release`	Live phone cursor move, L/R/double click, drag lock, and scroll	**PASS**
Live Phone API	`test_08_scenario_select_and_emergency_stop`	Live launch of Scenario 4 followed by immediate /api/scenario/stop	**PASS**
Live Phone API	`test_09_invalid_inputs_rejected_cleanly`	Live HTTP 400 validation for scenario ID 99 & unknown key name	**PASS**
Live Phone API	`test_10_cors_preflight_options`	Live OPTIONS request returns 204 No Content with CORS headers	**PASS**

Table 6.2: ESP32-S3 Firmware Resource Utilization and Real-Time Timing Metrics

Firmware / Hardware Resource Metric	Measured Value	Target Partition / Limit	Utilization / Engineering Notes
Compiled Flash Binary Size	851,520 bytes	1,310,720 bytes	64.9% of app partition (TinyUSB + Wi-Fi + PROGMEM UI)
Global Dynamic SRAM Allocation	59,496 bytes	327,680 bytes	18.1% of internal RAM (268,184 bytes free for heap/stack)
Boot-to-HID Enumeration Guard	3,500 ms	Configurable (config.h)	Prevents dropped keystrokes during initial USB bus reset
Emergency Stop Preemption Latency	<= 10 ms	10 ms smartDelay slice	Immediate scenario abort & dual HID releaseAll()

SECTION 07 | LIMITATIONS, CONCLUSION, FUTURE WORK & REFERENCES

7.1 Known Technical and Architectural Limitations

To maintain strict engineering transparency, four architectural boundaries of the current implementation are explicitly documented:

- **Unidirectional US-ASCII / QWERTY HID Assumption:** Standard USB HID Boot Keyboards transmit unacknowledged scancodes assuming a US-English QWERTY host layout. Non-ASCII Unicode characters are skipped by `KeyboardController::typeText()`, and the ESP32-S3 has no bidirectional visibility into which host window holds focus.
- **Cleartext Local HTTP Transport on Port 80:** Because mobile browsers block self-signed HTTPS certificates on dynamic private IPs (`192.168.4.1` or hotspot DHCP) without manual CA installation, the web controller uses unencrypted HTTP. While `X-Auth-Token` prevents unauthorized API execution, traffic is not encrypted against passive local sniffing.
- **Synchronous Single-Client HTTP Polling:** Arduino's built-in `WebServer` processes requests synchronously on a single thread. Touchpad motion events are therefore throttled to ~60 Hz (`16 ms`) on the smartphone client to avoid TCP queue saturation during rapid cursor swipes.
- **Global Scope of Win32 WH_KEYBOARD_LL Hook:** The Windows low-level hook (`keyboard.hook(suppress=True)`) intercepts all keyboard input globally at the OS level rather than filtering by a specific hardware USB Vendor/Product ID (`VID/PID`). During the 15-second blocking window, both BadUSB keystrokes and physical laptop keys are suppressed.

7.2 Technical Conclusion

This project successfully designed, implemented, debugged, and hardware-verified an end-to-end ESP32-S3 Phone-Controlled USB HID Keyboard, Mouse, and BadUSB Defense Platform. By unifying composite TinyUSB peripheral emulation (`USBHIDKeyboard` and `USBHIDMouse`), concurrent `WIFI_AP_STA` wireless networking, a deterministic 8-state scenario Finite State Machine, a four-tab mobile Single-Page Application, and a fail-safe `WH_KEYBOARD_LL` host keystroke firewall, the system bridges embedded IoT engineering and practical endpoint cybersecurity. Live verification on an ESP32-S3 (`COM5`) and an Android smartphone (`Vivo V50e`) confirmed 100% pass rates across all 22 automated simulation, unit, and live browser API tests.

7.3 Future Engineering Improvements

- **Asynchronous WebSockets & Embedded TLS (WSS/HTTPS):** Migrating from REST `POST` requests to persistent WebSocket frames over `ESPAsyncWebServer` with mbedTLS session encryption will reduce trackpad transport overhead and protect authentication tokens on shared wireless networks.
- **Per-Device Raw Input Attribution & Cadence Anomaly Detection:** Enhancing `src/firewall.py` with the Windows Raw Input API (`GetRawInputDeviceInfo`) and inter-keystroke timing analysis (<5 ms automated injection vs. >40 ms human typing cadence) will allow selective blocking of suspect USB HID peripherals while leaving the primary laptop keyboard unaffected.
- **Non-Volatile Scenario Storage & Multi-Locale Keymaps:** Storing user-defined educational scripts and international keyboard layouts (UK, DE, FR) inside ESP32-S3 LittleFS/NVS flash partitions will enable dynamic scenario authoring from the mobile browser without recompiling firmware.

7.4 References

[1] USB Implementers Forum (USB-IF), "Device Class Definition for Human Interface Devices (HID), Version 1.11," USB-IF Specification, 2001.

[2] USB Implementers Forum (USB-IF), "HID Usage Tables for Universal Serial Bus (USB), Version 1.4," USB-IF Technical Specification, 2022.

[3] K. Nohl and J. Lell, "BadUSB — On Accessories That Turn Evil," in Proc. Black Hat USA Security Briefings, Las Vegas, NV, USA, Aug. 2014.

[4] Espressif Systems, "ESP32-S3 Technical Reference Manual, Version 1.5," Espressif Systems, Shanghai, China, 2024.

[5] Espressif Systems, "Arduino Core for ESP32 — USBHIDKeyboard and USBHIDMouse API Reference (TinyUSB Stack)," Version 3.x, 2024.

[6] H. Thach, "TinyUSB: An Open-Source Cross-Platform USB Host/Device Stack for Embedded Microcontrollers," Technical Documentation, 2024.

[7] Microsoft Corporation, "SetWindowsHookExA Function and LowLevelKeyboardProc (WH_KEYBOARD_LL) Callback Reference," Win32 API Docs, 2024.

[8] BoppreH, "Python keyboard Library: Global Event Hook and Suppression Architecture for Windows and Linux," Version 0.13.5, PyPI, 2023.

VERIFICATION & ARCHIVAL SIGN-OFF: All firmware modules (badusb_firmware/), host keystroke firewall defenses (src/firewall.py), automated test suites (22/22 passed), and mobile UI views have been compiled, flashed to physical ESP32-S3 hardware (COM5), and verified end-to-end on Android (Vivo V50e). All sensitive credentials, tokens, and device serial numbers are redacted in compliance with institutional cybersecurity guidelines.